

Marcador Electrónico para Natación —

Prototipo General

Repositorio general / prototipo inicial del marcador electrónico de natación. Contiene la **primera versión** del firmware CircuitPython para el **Adafruit MatrixPortal S3 (ESP32-S3)**: una implementación más sencilla que permite mostrar texto controlado por MQTT en una matriz LED de **64x32**.

Este repo es la versión temprana del proyecto. La versión completa y funcional del firmware vive en `marcador-electronico-para-natacion-circuitpython`, y la interfaz web de control en `marcador-electronico-para-natacion-frontend`.

Configuración (`settings.toml`)

Edita el archivo `settings.toml` en la raíz del dispositivo:

```
CIRCUITPY_WIFI_SSID = "red_wifi"
CIRCUITPY_WIFI_PASSWORD = "contraseña_wifi"

MQTT_BROKER = "tu-broker.hivemq.cloud"
MQTT_PORT = 8883
MQTT_USERNAME = "usuario_mqtt"
MQTT_PASSWORD = "contraseña_mqtt"
MQTT_SSL = "true"
```

Variable	Descripción
<code>CIRCUITPY_WIFI_SSID</code>	Nombre de la red WiFi
<code>CIRCUITPY_WIFI_PASSWORD</code>	Contraseña de la red WiFi
<code>MQTT_BROKER</code>	Dirección del broker MQTT (ej. HiveMQ Cloud)
<code>MQTT_PORT</code>	Puerto MQTT (<code>1883</code> sin SSL, <code>8883</code> con SSL)
<code>MQTT_USERNAME</code>	Usuario del broker MQTT

Variable	Descripción
MQTT_PASSWORD	Contraseña del broker MQTT
MQTT_SSL	"true" para usar SSL/TLS, "false" para conexión sin cifrar

Nota de seguridad: no subas credenciales reales a repositorios públicos. Borra los secretos antes de hacer commit.

Funcionamiento

Ciclo principal (`code.py` → `utils/main.py`)

1. Conecta a WiFi usando las credenciales de `settings.toml`
2. Configura y conecta el cliente MQTT
3. Entra en un bucle que: - Mantiene la conexión MQTT (`mqtt_client.loop()`) - Lee la dirección de pantalla (`ARRIBA` , `ABAJO` , `IZQUIERDA` , `DERECHA`) y mueve el texto en consecuencia - Muestra el mensaje recibido en la matriz con su color y escala

Pantalla

La matriz se inicializa en `utils/screen.py` :

```
matrix = Matrix(width=64, height=32)
```

El texto se renderiza con la función `show_text(texto, x, y, color, escala)` sobre la fuente de terminal.

Tópicos MQTT

Suscripciones (el dispositivo escucha)

Tópico	Formato	Descripción
esp32s3/controltext	texto	Mensaje a mostrar en la matriz
esp32s3/screendirection	ARRIBA / ABAJO / IZQUIERDA / DERECHA	Mueve la posición del texto
esp32s3/screencolor	#RRGGBB	Cambia el color del texto
esp32s3/sizechar	small / medium	Cambia la escala del texto

Publicaciones (el dispositivo envía)

Tópico	Descripción
esp32s3/status	online / offline (mensaje de voluntad)
esp32s3/sensor	Datos simulados de sensor: temperatura,humedad

Archivos del proyecto

```

├─ code.py                # Punto de entrada (llama a utils.main.main())
├─ settings.toml          # Configuración WiFi y MQTT
├─ boot_out.txt           # Información del dispositivo CircuitPython
├─ lib/                   # Librerías CircuitPython
│   ├── adafruit_matrixportal/
│   ├── adafruit_minimqtt/
│   ├── ...
│   └─ utils/             # Código fuente de la aplicación
│       ├── main.py        # Bucle principal
│       ├── mqtt_client.py # Cliente MQTT con callbacks
│       └─ screen.py       # Control de la matriz LED
└─ sd/                   # Directorio para tarjeta SD (sin usar)

```

Relación con los otros repositorios

- **General** (este repo): prototipo inicial del firmware.
- **CircuitPython** (`marcador-electronico-para-natacion-circuitpython`): firmware completo y funcional (cronómetro, RTC, records, puntuaciones).

- **Frontend** (`marcador-electronico-para-natacion-frontend`): interfaz web de control vía MQTT.